Vision-Measured Spatial Queues for Storage-Aware Traffic Signal Control

Master Guide (Hinglish): talk ka flow, har detail ke peeche ki wajah, aur deep questions ke answers

- Is guide ko kaise use karein
  - Project 60 seconds mein
- Part A — Talk, stage by stage
  - A1. Problem (2 min) — slides 1–5
  - A2. Base paper aur uski limitation (2 min) — slides 6, 10
  - A3. Nayi research aur gap (2 min) — slides 7–11
  - A4. Measurement — worked example (4 min) — slides 15–16
  - A5. Measurements se signal kaise chalta hai (2 min) — slide 15
  - A6. Video se control evaluate kyun nahi ho sakta, aur audit (1.5 min)
  - A7. Fair test (1.5 min) — slide 14
  - A8. Study 1 — pehla enhancement aur woh fail kyun hua (1.5 min) — slide 17
  - A9. Study 2 — final enhancement (2 min) — slides 18–19
  - A10. Results — yahi ablation bhi hai (3 min) — slides 20–25
  - A11. Close (1.5 min) — slides 24–27
- Part B — Talk ke peeche ki saari knowledge
  - B1. Problem ki vocabulary
  - B2. Data
  - B3. Detection — YOLOv8m
  - B4. Tracking — ByteTrack aur track cache
  - B5. Geometry — regions aur queue axis
  - B6. Vehicle ko approach assign karna
  - B7. Density D aur queue Q
  - B8. Stopped test
  - B9. Road par position, p
  - B10. Queue tail X
  - B11. Forecasts F aur S, aur blind-spot proof
  - B12. Score (study 1)
  - B13. Controller aur green timing
  - B14. Audit — pehle kya galat tha, aur kaise pakda
  - B15. Simulation testbed (SUMO)
  - B16. Statistics — farak real hai, yeh kaise decide hota hai
  - B17. Study 1 — design, result, wajah
  - B18. Study 2 — final enhancement
  - B19. Papers — har paper kya kehta hai, kya liya
  - B20. Kya mera hai aur kya borrowed
  - B21. Limitations aur future work
- Part C — Cross-question bank
- Part D — Reference
  - D1. Yaad karne wale numbers
  - D2. Har formula kahan se aaya
  - D3. Code map
  - D4. Live dikhane ke commands
  - D5. Short glossary

Is guide ko kaise use karein

- **Part A** = presentation ka flow. Story usi order mein jaise aap sir ko sunaoge, aur har stage ke end mein ek “bridge” line jo agle stage se jodti hai. Sabse pehle yeh yaad karo.
- **Part B** = har stage ke peeche ki poori knowledge: har component ke liye **kya** hai, **kyun** aise kiya, **kaise** kaam karta hai (numbers ke saath), code mein **kahan** hai, aur idea **kahan se aaya**. Jab sir “why?” ya “how exactly?” poochhein, answer yahan hai.
- **Part C** = cross-question bank: strict examiner jo deep questions poochhta hai, unke short answers.
- **Part D** = reference: yaad karne wale numbers, formulas ke source, code map, commands, glossary.

Pehle Part A bina notes ke bolna seekho. Phir Part B ek baar mein ek section padho, aur Part C se khud ko test karo.

Project 60 seconds mein

> *Traffic signal decide karta hai ki kaunsi road ko green milega aur kitni der ke liye. Raza et al. (2025) yeh camera se karte hain, lekin woh sirf vehicles **count** karte hain. Nayi research (Mohajerpoor et al., IEEE T-ITS 2023) dikhati hai ki queue ko road se bahar overflow hone se rokne ke liye queue ki **position** chahiye — aur woh use ek model se estimate karte hain. Main woh position seedha video se **measure** karta hoon: YOLOv8 vehicles detect karta hai, ByteTrack unko follow karta hai, aur mera method batata hai ki stopped vehicles ki chain kahan khatam hoti hai (X) aur kidhar ja rahi hai (S). Pehle maine X aur S ko ranking score mein daala — pre-registered simulation ne dikhaya ki isse kuch nahi badalta, aur main uski wajah samjha sakta hoon. Phir maine S ko green timing mein storage limit ki tarah use kiya: capacity ke paas yeh standard actuated control ke comparison mein spillback 20–26% kam karta hai, same delay par; aur capacity se upar yeh fail hota hai. Main dono results report karta hoon.*

Part A — Talk, stage by stage

Har stage mein: **Bolo** (sir ko kya kehna hai), **Dikhao** (slide ya figure), **Bridge** (agle stage mein jaane wali line). Total lagbhag 20 minute.

A1. Problem (2 min) — slides 1–5

Bolo: “Ek signalised junction baar-baar decide karta hai ki agla green kis approach ko milega aur kitni der ke liye. Fixed-time plans traffic dekhe bina same timing repeat karte hain. Adaptive control traffic measure karta hai. Loop detector road ka sirf ek point dekhta hai; camera poori road dekhta hai, isliye woh measure kar sakta hai ki *queue kahan khatam hoti hai*. Yeh important hai kyunki jab queue apni road ke start tak pahunch jaati hai — isko spillback kehte hain — toh peeche ka traffic block ho jaata hai.”

Dikhao: demo frame (slide 3): boxes, IDs, approaches, queue tails.

Bridge: “Toh mera question hai: camera ko kya measure karna chahiye, aur kya isse control better hota hai?”

A2. Base paper aur uski limitation (2 min) — slides 6, 10

Bolo: “Raza et al. 2025 YOLO se vehicles count karte hain, har vehicle ko uske PCE se weight karte hain — ek truck teen cars ke barabar — aur sabko jod ke density D banate hain. Jis road ki density sabse zyada, use green milta hai, 40, 60 ya 120 second ke liye. Mera analysis: unka state ek count hai. Woh nahi bata sakta ki queue kahan khatam hoti hai ya har road kitni lambi hai. 60 metre ki road par 10 cars aur 150 metre ki road par 10 cars — dono same dikhti hain, jabki pehli road lagbhag full hai.”

Bridge: “Nayi research kehti hai ki yahi position information matter karti hai.”

A3. Nayi research aur gap (2 min) — slides 7–11

Bolo: “Li et al. 2025 queue ka *profile* space mein estimate karte hain, lekin unhe connected vehicles ka GPS data aur offline solver chahiye. Wei et al. 2025 queues ko *predict* karke spillback se pehle action lete hain, lekin maan lete hain ki queue ki state pata hai. Mohajerpoor et al. 2023 ek over-saturated junction ko spillback constraint ke saath control karte hain — queue link length ke β guna se zyada nahi honi chahiye — aur ek penalty jo queue ke road ke end ke paas pahunchne par tezi se badhti hai. Lekin woh queue ki position shockwave model se *estimate* karte hain. Gap yeh hai: camera wale controllers sirf count karte hain; spillback-aware controllers dekh nahi sakte. Mera idea: queue ki position camera se measure karo.”

Bridge: “Pehle mujhe isse reliably measure karna tha. Ek real frame par dikhata hoon kaise.”

A4. Measurement — worked example (4 min) — slides 15–16

Ek real frame follow karo (busy clip, frame 2036, West road; har number Part B7–B11 mein hai):

1. “YOLOv8m har vehicle ka box deta hai; ByteTrack har vehicle ko ek ID deta hai jo frames ke across same rehti hai. Main box ka bottom-centre leta hoon — jahan wheels road ko chhute hain.”
2. “Maine har road par stop line se peeche ki taraf ek line draw ki. Car ki position p matlab woh us line par kitni door hai, fraction mein: car #2 p = 0.57 par hai.”
3. “Car *stopped* hai agar pichhle ek second mein woh apni box height ke 0.2 se kam hili. Apne size se divide karne se perspective cancel ho jaata hai; ek second box ki jitter ko average kar deta hai; tracking batata hai ki yeh wahi car hai.”
4. “Stop line se peeche chalte hue, stopped cars ki chain follow karo jab tak gap do car-length se kam ho. Aakhri car queue ka end hai: X = 0.57.”
5. “S, X ke trend se 5 second aage project karta hai: yahan S = 0.55 — steady.”

“In fixes se false stops 4.28 se 0.76 per vehicle ho gaye.”

Bridge: “Ab mere paas har road aur har frame ke liye D, Q, X, S hain. Controller inka use kaise karta hai?”

A5. Measurements se signal kaise chalta hai (2 min) — slide 15

Bolo: “Do decisions. *Agli road kaunsi:* har road ka score banta hai, sabse zyada wala jeet-ta hai, aur starvation guard ensure karta hai ki koi road teen baar se zyada skip na ho. *Kitni der:* ya toh Raza ke bands — score ke hisaab se 30, 45 ya 60 s — ya actuated timing, jo green tab khatam karti hai jab stop-line zone 2 second tak khaali rahe; kabhi 10 s se kam nahi, 60 s se zyada nahi.”

Bridge: “Yeh help karta hai ya nahi, iske liye fair test chahiye — aur video woh nahi de sakta.”

A6. Video se control evaluate kyun nahi ho sakta, aur audit (1.5 min)

Bolo: “Video mein cars asli signal follow karti hain jo film hua tha, mera simulated signal nahi — isliye mera controller unki waiting change nahi kar sakta. Pehle ek result tha — video par 21% zyada throughput — jo artefact nikla: ek control version jismein mera naya term zero kar diya, usne bhi exactly same number diya. Isliye main video se *measurement* test karta hoon, aur simulator se *control*.”

Bridge: “Yeh raha simulator ka setup.”

A7. Fair test (1.5 min) — slide 14

Bolo: “SUMO ek open-source traffic simulator hai jismein vehicles mere signal ki wajah se rukte aur chalte hain. Ek virtual camera D, Q, X, S ko video pipeline ki same definitions se compute karta hai. Har method ko same random traffic milta hai, isliye main paired differences ko 95% confidence intervals ke saath compare karta hoon. Har setting alag validation runs par fix hui aur final test se pehle protocol mein likh di gayi.”

Bridge: “Mera pehla enhancement X aur S ko ranking score mein daalna tha.”

A8. Study 1 — pehla enhancement aur woh fail kyun hua (1.5 min) — slide 17

Bolo: “Yeh ek ablation hai: fixed-time; Raza-style; plus queue; plus count forecast; plus mera S; aur ek null control. X ya S jodne se delay maximum aadha second badla, aur chuni hui road chaalis mein se sirf ek run mein badli. Wajah: jab green khatam hota hai, jis road ki queue sabse lambi hai wahi road sabse zyada cars wali bhi hoti hai — toh X roads ko waise hi rank karta hai jaise count. Jo cheez sach mein matter ki woh timing thi: sabse mushkil scenario mein actuated timing ne delay 300 se 34 second kar diya. Seekh: position information sirf *timing* mein matter kar sakti hai, aur *har road ki apni length* ke relative.”

Bridge: “Mohajerpoor bhi exactly aise hi use karte hain, toh maine agla wahi banaya.”

A9. Study 2 — final enhancement (2 min) — slides 18–19

Bolo: “Raza ke score mein ek extra term aur ek timing rule. Term hai λ(1/(1.1 − S) − 1/1.1): chhoti queue ke liye lagbhag zero, aur jaise queue road ke end ke paas aati hai, bahut bada. Rule: 20 second green ke baad, agar kisi waiting road ka S 0.85 tak pahunch jaaye toh current green jaldi khatam karo. Dono ka *shape* Mohajerpoor ka hai; mera hai — S ko model ki jagah camera se measure karna, unke optimisation ko camera controller ke andar ek online rule banana, aur saare experiments.”

Bridge: “Maine isse standard aur published baselines ke against paanch traffic levels par test kiya.”

A10. Results — yahi ablation bhi hai (3 min) — slides 20–25

Bolo:

- “Do junctions — saari roads 150 m, ya ek side street 60 m — paanch demand levels, aath methods, bees seeds. Rows ek ablation hain: Raza → plus actuated timing → plus barrier → plus protection → camera vs count.”
- “Raza ke against, das mein se nau conditions mein 34 se 87 second kam delay — zyada tar actuated timing ki wajah se.”
- “Capacity ke paas, 3000 vehicles per hour: standard actuated control se 20 se 26 percent kam spillback, same delay par. Chhoti road par camera ki measured queue count ko 115 second spillback se beat karti hai.”
- “Capacity se upar, 3600: yeh fail hota hai — delay lagbhag double, kyunki har road full ke paas hai aur rule baar-baar greens kaat-ta hai. Mohajerpoor exactly isi regime ki warning dete hain.”
- “Light traffic par capacity-aware max pressure sabse best method hai; woh bhi main report karta hoon.”

A11. Close (1.5 min) — slides 24–27

Bolo: “Contributions: video se calibration-free queue measurement; uspar bana ek storage-aware rule; aur evidence ki yeh kahan help karta hai aur kahan fail hota hai. Limitations: X sirf visible road cover karta hai aur door ki cars miss hoti hain; ek hi junction; abhi hand-labelled accuracy nahi hai. Aage: jab har road full ho tab protection band karna; pixels ko metres mein convert karna; door ki roads par better detection; do junctions ka corridor.”

Last line: “Same camera measurement agli road chunne ke liye bekaar thi, aur capacity ke paas storage limit ki tarah useful thi. Information controller mein *kahan* jaati hai, yeh *kya* measure kiya usse zyada matter karta hai.”

Part B — Talk ke peeche ki saari knowledge

B1. Problem ki vocabulary

Term	Matlab
Approach	Junction mein aane wali ek road (yahan North, East, South, West)
Phase / green	Jitni der ek approach ko chalne ki permission hai; yahan ek time par ek approach
Yellow	Greens ke beech 3 s; junction ke liye lost time
Delay	Free driving ke comparison mein vehicle ka jo time waste hua (SUMO “time loss”) + full road mein enter karne ke liye wait
Storage	Ek approach ke paas queue ke liye jitni road hai
Spillback	Queue road ke upstream end tak pahunch gayi, naye vehicles enter nahi kar sakte
Capacity	Junction ek ghante mein maximum kitna traffic serve kar sakta hai
Under / near / beyond capacity	Demand capacity se kaafi kam, capacity ke paas, ya capacity se upar
PCE	Passenger Car Equivalent: car 1, motorcycle 0.5, bus aur truck 3

B2. Data

- **Kya:** City of Bellevue Traffic Video Dataset (research ke liye public) ke teen clips, 116th Ave NE / NE 12th St par ek fisheye camera: 107 s, 240 s aur 240 s, constant 30 frames per second, 1280 × 720.
- **Yeh data kyun:** ek real, public intersection camera jo chaaron approaches dekhta hai.
- **Limit:** ek junction, din ka time, ek camera angle; door wali roads (North, East) image mein chhoti hain.
- **Kahan:** `videos/`, `data/annotations/videos.json`.

B3. Detection — YOLOv8m

- **Kya:** YOLO (“You Only Look Once”) ek one-stage object detector hai: convolutional network ka ek pass boxes, classes aur confidence deta hai. v8m = version 8, medium size.
- **Settings:** COCO-pretrained weights, classes car, motorcycle, bus, truck; confidence ≥ 0.3.
- **Training:** koi nahi. COCO dataset mein yeh classes pehle se hain. Contribution detector nahi hai — detections se jo compute hota hai woh hai.
- **m kyun, n kyun nahi:** door ki chhoti cars ka recall. Ek test frame par YOLOv8n ne North ki ~9 queued cars mein se 2 aur East ki ~10 mein se 0 dhoondhi; YOLOv8m ne 4

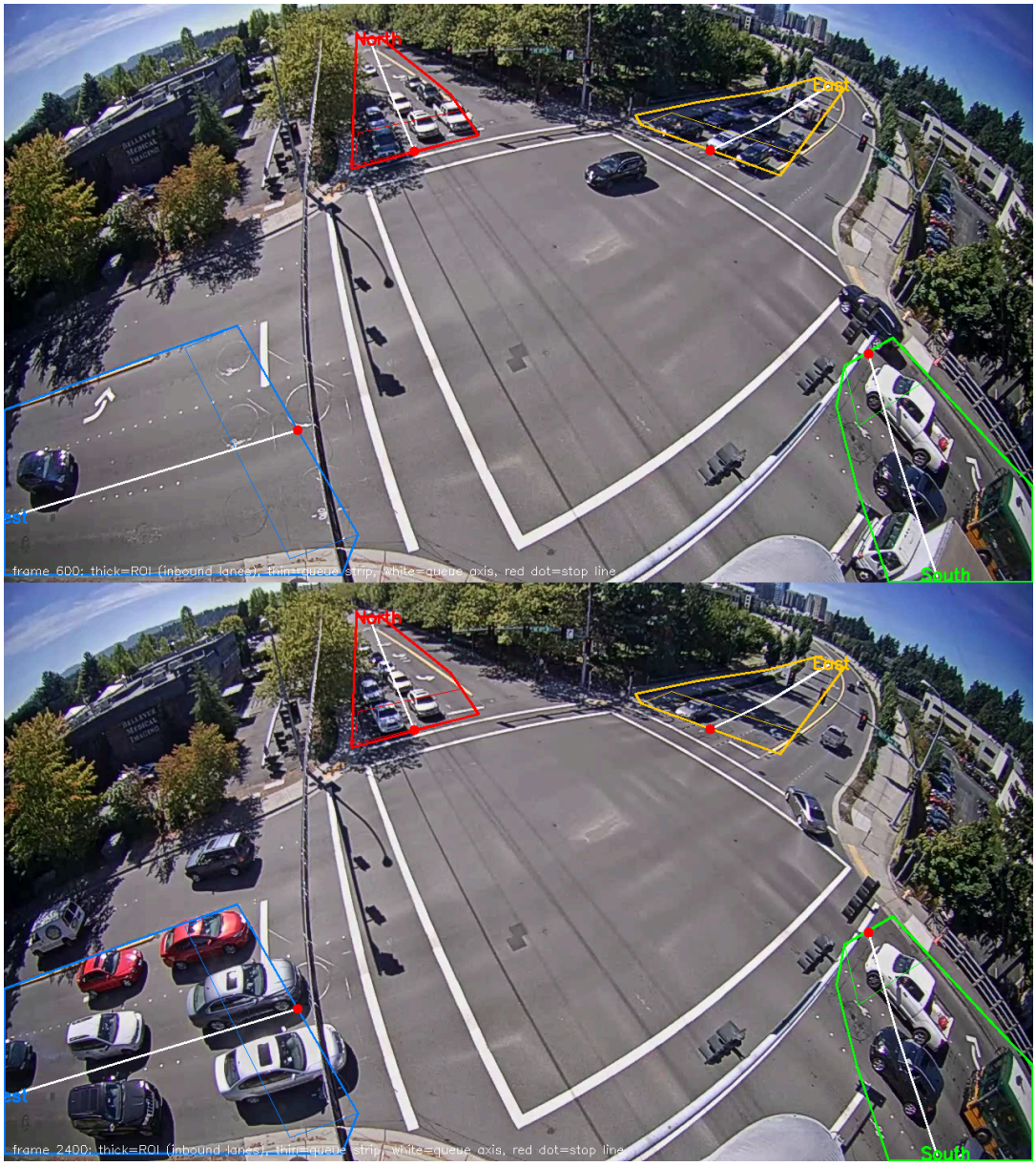
aur 3.
- **Kahan:** `src/tracking.py`; weights `models/yolov8m.pt`.
- **Source:** Ultralytics YOLOv8 (Jocher et al. 2023), wahi family jo Raza use karte hain.

B4. Tracking — ByteTrack aur track cache

- **Kya:** ByteTrack har frame ke boxes ko existing tracks se overlap aur motion ke basis par jodta hai, aur har vehicle ko ek ID deta hai jo same rehti hai. Iska key idea: *low-confidence* boxes ko bhi existing track continue karne ke candidate ki tarah rakhna, jisse partial occlusion mein bhi ID nahi tootti.
- **Tracking kyun zaroori:** "stopped" ka matlab hai "yahi vehicle ek second mein mushkil se hila"; iske liye pata hona chahiye ki ek second pehle kaunsa box yahi vehicle tha.
- **Track cache:** detection aur tracking har clip par ek baar chalta hai; tracks save hote hain aur har experiment unhe replay karta hai — toh saare controllers ko bilkul same detections milti hain, aur runs fast aur repeatable hote hain.
- **Kahan:** `src/tracking.py` (Ultralytics `persist=True`), `src/track_cache.py`, `results/track_cache/`.
- **Source:** Zhang et al., ByteTrack, ECCV 2022.

B5. Geometry — regions aur queue axis

- **Kya:** har approach ke liye, is camera ke liye ek baar haath se draw kiya (`config/bellevue_116th_v2.json`):
  - ROI — us road ki inbound lanes par polygon;
  - queue strip (`queue_region`) — stop line ke theek peeche chhota polygon;
  - queue axis — stop line se road ke saath peeche view ke edge tak ek polyline.
- **Dobara kyun draw kiya (audit finding W7):** original regions do legs aur dono directions cover karte the, queue regions stop line par nahi the, aur do automatic axes ulti direction mein the. Matlab purana "queue reach" stop line se measure hi nahi ho raha tha. v2 geometry ne yeh fix kiya.
- **Axis lengths:** North 149 px, East 168 px, South 290 px, West 381 px.
- **Kahan:** `src/lane_analysis.py`; picture `report/geometry_v2.png`.



Approach geometry v2: ROI, queue strip, queue axis (white), stop line (red dot).

B6. Vehicle ko approach assign karna

- **Kya:** vehicle us approach ka hai agar uska **reference point** — box ka bottom-centre, ((x1 + x2)/2, y2) — us approach ke ROI ke andar hai (point-in-polygon test). Agar point queue region mein hai toh woh “queue strip mein” hai.
- **Bottom-centre kyun:** wahan vehicle road ko chhoota hai. Lambe truck ke box ka centre road se upar hawa mein hota hai aur galat region mein gir sakta hai.
- **Kahan:** `src/lane_analysis.py` (`ApproachAssigner`).

B7. Density D aur queue Q

- **D** = (ROI ke andar vehicles ka Σ PCE) / `saturation_count`, maximum 1. `saturation_count` = visible ROI mein kitne vehicles aa sakte hain (West 12, North 12, East 10, South 6).
- **Q** = (queue strip mein vehicles) / `queue_capacity` (4, South par 2), maximum 1.
- **Normalise kyun:** har measure ko 0–1 scale par laane ke liye taaki unhe jod sakein.
- **Source:** D Raza ka Eq. 1 hai; normalisation is project ka hai.

- **Example (frame 2036, West):** 5 cars → D = 5/12 = 0.417; strip mein 1 car → Q = 1/4 = 0.25.

B8. Stopped test

$$\text{speed} = \frac{\text{pichhle 1 s mein reference point kitna hila}}{1\,\text{s} \times h}, \qquad \text{stopped} \;\Leftrightarrow\; \text{speed} < 0.2$$

- h = vehicle ki box height pixels mein: image mein uska apna size.
- **h se divide kyun:** door ki car chhoti hai, toh ek metre chalne par kam pixels cover hote hain; fixed pixel threshold door ki chalti cars ko "stopped" bol dega. Apne size se divide karne se paas aur door ki cars same scale par aa jaati hain — 0.2 box heights per second matlab 5 m car ke liye lagbhag 1 m/s.
- **Ek second kyun:** boxes har frame thoda jitter karte hain; purana test (*pichhle frame* se 2 px se kam hila) baar-baar flip hota tha. Ek second mein jitter average ho jaata hai.
- **Effect:** busy clip par stops per vehicle 4.28 → 0.76 (audit W5).
- **Example:** car #3 ek second mein 16.5 px hili, h = 92 px → 0.179 → stopped. Car #4 105.7 px hili, h = 80 → 1.32 → moving.
- **Kahan:** `src/traffic_metrics.py`, `_is_stopped_windowed`.

B9. Road par position, p

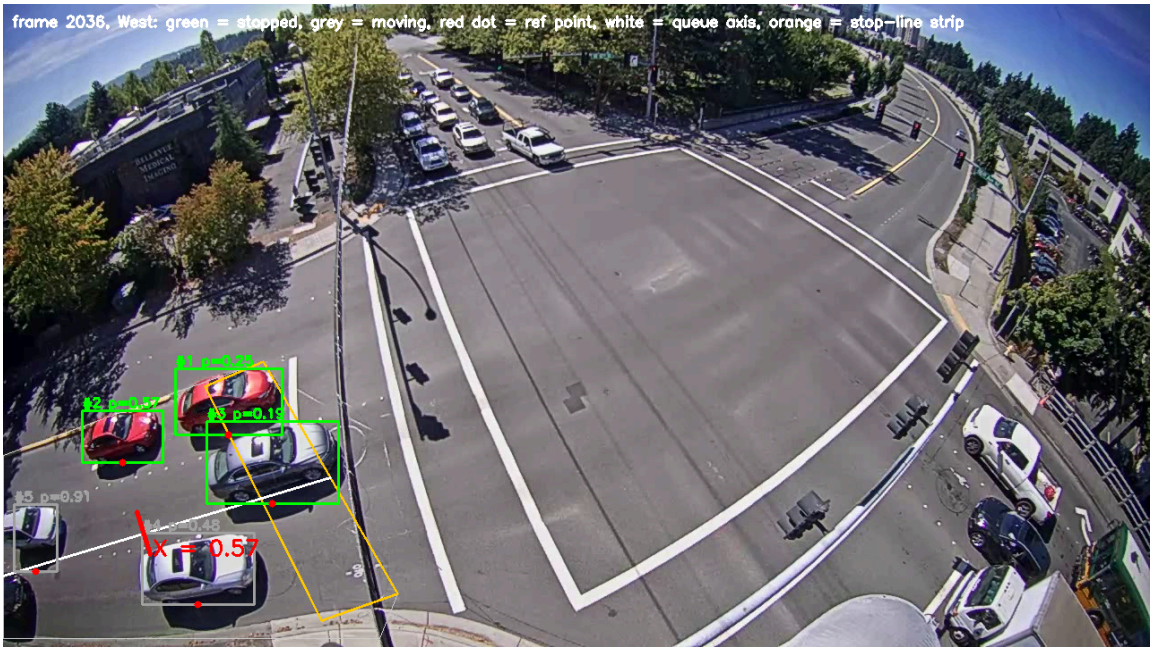
$$t = (\text{ref} - A) \cdot u, \qquad p = t/L$$

- A = axis ka pehla point (stop line); u = axis ki direction ka unit vector, upstream; L = axis length pixels mein. Multi-segment axis mein point ko nearest segment par project karke arc length lete hain.
- **Example (West):** A = (365, 530), u = (−0.957, 0.289), L = 381.2 px. Car #2 (133, 513) par: t = 222.0 − 4.9 = 217.1 → p = 0.570.
- **Matlab:** p = 0.57 matlab stop line se *visible* road ke end tak ka 57% raasta. Camera calibration ki zaroorat nahi, lekin p metres mein nahi hai (perspective door ke hisse ko compress karta hai) aur sirf visible hisse ke relative hai.
- **Kahan:** `src/lane_analysis.py`, `_polyline_fraction`.

B10. Queue tail X

- **Rule:** stopped vehicles lo, p ke hisaab se sort karo. Pehle vehicle se start karo jo queue strip ke andar hai (ya stop line se apne allowance ke andar). Tab tak aage badho jab tak p_next − p_previous ≤ agle vehicle ka allowance. X = aakhri member ka p; koi nahi toh 0.
- **Allowance** = 2 × (vehicle ki road ke saath length) / L, jahan road ke saath length = $|w \cdot u_x| + |h \cdot u_y|$ (box width w, height h, road direction u).
- **Contiguous kyun:** parked car ya road par door ki koi detection queue ka hissa nahi hai; chain rule ke bina aisi ek car X ko jump karwa deti hai. Purana version 8% frames tak mein road ke 30% se zyada jump karta tha; ab ≤ 1.1%.
- **Box height ki jagah road ke saath length kyun:** West leg par cars side se dikhti hain, height se do guna chaudi; "do box heights" ek car + gap se chhota tha aur real queues tod deta tha.
- **Example (frame 2036, West):** #3 p = 0.185 (strip mein, start) → #1 p = 0.250 (gap 0.065 ≤ 0.710) → #2 p = 0.570 (gap 0.320 ≤ 0.540) → **X = 0.570**.

- **Kahan:** `src/traffic_metrics.py` mein `queue_tail_reach_scaled`, `_extent_along`.
- **Source:** is project ka design; spatial queue profile ka idea Li et al. se.



Ek real frame: green = stopped, grey = moving, red dot = reference point, white = queue axis, orange = stop-line strip, red bar = X.

B11. Forecasts F aur S, aur blind-spot proof

$$F = \text{clamp}(Q + \text{slope}_Q \times 3\,\text{s}), \qquad S = \text{clamp}(X + \text{slope}_X \times 5\,\text{s})$$

- **Slope:** pichhle 2.5 s (75 frames) ki values par least-squares straight line; uska slope per second. Bahut saare points hone se ek noisy frame ka asar nahi padta. (Pehle 0.5 s window aur 5 s horizon tha, jo noise ko 10 guna kar deta tha.)
- **clamp** result ko [0, 1] mein rakhta hai.
- **S ka matlab:** 5 s baad queue tail kahan hogi. Steady queue ke liye S = X, badhti queue ke liye X se upar, clear hoti queue ke liye neeche. S = 1: queue visible road bharne wali hai. Yeh *local* risk hai; camera downstream nahi dekh sakta.
- **Blind-spot proof (count kyun nahi, X kyun):** strip mein 4 cars aati hain. Full hone ke baad har frame Q = 1, slope 0, toh F = 1 + 0 = 1 — “steady” — chahe strip ke peeche kitni bhi tezi se cars judti rahein. F sirf clearing dikha sakta hai. X peeche khisakta rehta hai, toh S badhta rehta hai.
- **Honest note (audit W3):** real footage par Q = 1 sirf 0.6% frames tak hua, toh yeh blind spot mathematically sahi hai lekin clips mein kam hua; ise simulation mein test kiya.
- **Example:** X 0.570 par steady, slope −0.0035/s → S = 0.570 − 0.0175 = 0.552.
- **Kahan:** `src/traffic_metrics.py` mein `QueuePredictor`.
- **Source:** predicted queue par action ka idea Wei et al. ka; least-squares line extension standard statistics.

B12. Score (study 1)

$$\text{Score}_i = (1 - \omega - \rho - \psi)\,[\alpha D_i + (1 - \alpha) Q_i] + \omega F_i + \psi X_i + \rho S_i$$

- α = D aur Q ke beech ka weight (0.5); ω, ψ, ρ = F, X, S ke weights. Proposed S4: ω = 0.3, ρ = 0.3, ψ = 0 → Score = 0.4(0.5D + 0.5Q) + 0.3F + 0.3S.
- **Weighted average kyun:** har term [0, 1] mein hai, toh score bhi [0, 1] mein rehta hai aur green-time bands apply hote hain.
- **Weights haath se set kiye, aur jaan-boojh ke tune nahi kiye**, taaki test data par fit na ho.
- **Example:** West = 0.4 × 0.333 + 0.3 × 0.630 + 0.3 × 0.552 = 0.488 (us frame par sabse zyada).
- **Kahan:** `src/traffic_metrics.py` mein `compute_score_weighted`.

B13. Controller aur green timing

- **Agli road kaunsi:** score ka argmax, ek **starvation guard** ke saath: jo approach lagatar 3 baar skip hua, use agla green milta hai. Raza mein bhi same mechanism hai (Green Denial Counter).
- **Raza-style bands:** score < 0.3 → 30 s; < 0.6 → 45 s; warna 60 s (Raza: 40/60/120 s). Green kabhi jaldi khatam nahi hota.
- **Actuated timing:** green kam se kam 10 s; jab stop-line strip 2 second (passage time) tak khaali rahe toh khatam; maximum 60 s. Yeh standard practice hai (FHWA Signal Timing Manual). 2 s passage time testing se aaya: strip khaali hote hi green khatam karne se greens discharge ke beech mein kat jaate the.
- **Signal sequence:** GREEN → 3 s YELLOW → agla approach.
- **Kahan:** `src/signal_controller.py` (`AdaptiveController`, `PhaseSequencer`).

B14. Audit — pehle kya galat tha, aur kaise pakda

Finding	Kya galat tha	Kaise dikhaya	Fix
W1	Video “throughput” *asli* filmed light se agreement measure karta hai, control quality nahi	Served vehicles = recorded departures jo simulated green mein pade; “best” run ne poore clip North green rakha	Control sirf closed-loop simulation mein test
W2	+21% (S3 → S4) weight ka artefact tha	Do cheezein badli (S add hua aur base weight 0.7 → 0.4); NULL control (S = 0) ne bhi exactly 158 diya, S4 jitna	Har experiment mein null control
W3	Footage par Q almost kabhi saturate nahi hua	Q = 1 sirf ≤ 0.6% frames mein	Blind spot simulation mein test
W4	Ek decision, ek clip	S3 aur S4 sirf ek truncated phase mein alag	20 seeds, 9 scenarios
W5	X aur S par noise haavi	4.3 stops per vehicle; 8% frames mein X jump	1 s size-scaled stopped test; contiguous tail; 2.5 s slope
W6	Raza ka “reproduction” zyada claim	Alag bands, lane-priority weights nahi	Har jagah “Raza-style”

Finding	Kya galat tha	Kaise dikhaya	Fix
W7	Geometry queue define nahi karti thi	Regions do legs cover karte; axes ulti direction	v2 geometry dobara draw

Kahan: `AUDIT_REPORT.md`, scripts `audit/` mein.

B15. Simulation testbed (SUMO)

- **Kya:** SUMO (Simulation of Urban MObility), open-source, microscopic: har vehicle simulate hota hai aur signal follow karta hai. Python se `libsumo` ke through har 0.5 s chalaya.
- **Junction:** chaar arms, har mein do lanes, seedha jaana, ek time par ek approach green, 3 s yellow. Study 1: saare approaches 150 m. Study 2: ek junction jismein East/West 60 m.
- **Demand:** random (Poisson) arrivals, 5% heavy vehicles. Study 1: 9 scenarios (light, medium, heavy, over-saturated, unequal, unequal over-saturated, surge, growing, aur ek video ke measured shares se calibrated). Study 2: 1200–3600 veh/h, split N 35%, S 30%, E 20%, W 15%. 1800 s demand, pehle 300 s warm-up ke roop mein hata diye.
- **Virtual camera:** simulated vehicles ki position aur speed se D, Q, X, S compute karta hai, video pipeline ke *same functions* se. Noise mode: door ke 30% tak vehicles miss aur 2 m position jitter.
- **Measures:** mean delay per vehicle (time loss + insertion delay); blocked-entry seconds (jo vehicles enter nahi kar paaye kyunki queue upstream end tak thi) = local spillback.
- **Simulation kyun:** closed loop hai — vehicles *mere* signal par react karte hain — jo recorded video mein possible nahi.
- **Kahan:** `sim/scenario.py`, `sim/sensor.py`, `sim/closed_loop.py`, `sim/study2.py`.

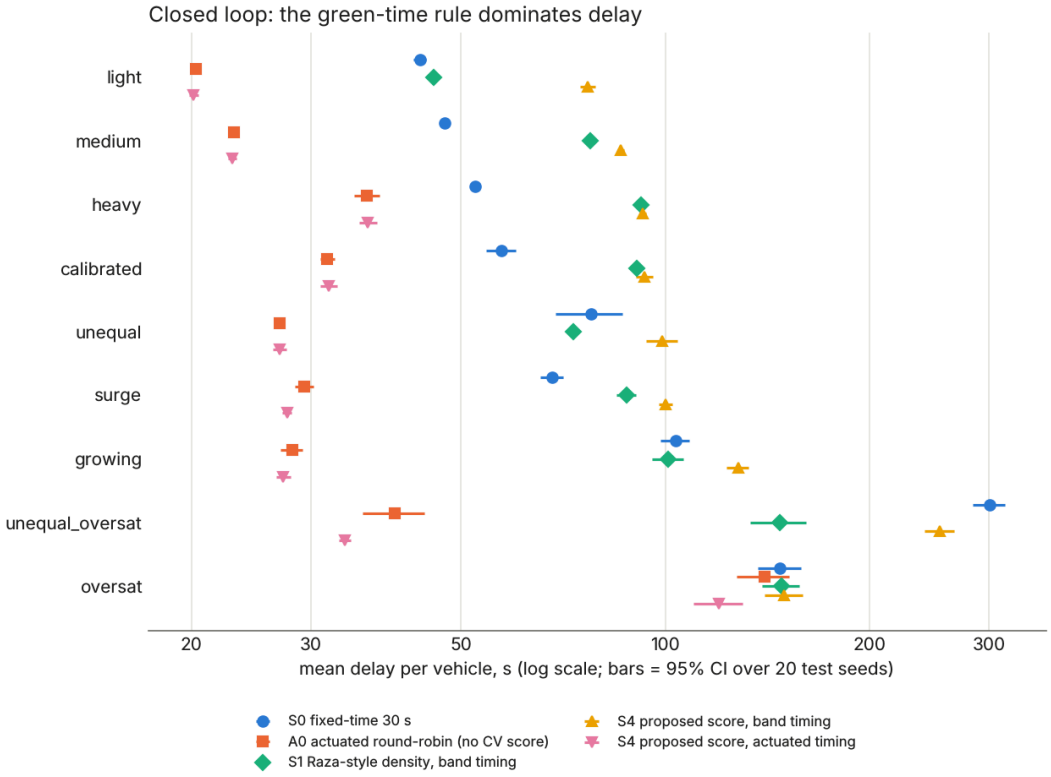
B16. Statistics — farak real hai, yeh kaise decide hota hai

- **Seed:** woh number jo random traffic fix karta hai. Har method same seeds par chalta hai, toh sabko bilkul same traffic milta hai (common random numbers).
- **Paired difference:** same seed par method A minus method B, 20 seeds ka average. Pairing seed-to-seed ka bada variation hata deti hai.
- **95% confidence interval:** mean ± t × SD/√n (Student t, 19 degrees of freedom, t ≈ 2.09). Agar interval mein 0 nahi aata, toh farak real maana jaata hai.
- **Validation vs test seeds:** settings validation seeds par chuni (study 1: 0–3; study 2: 0–9); results test seeds par report (100–119; 200–219), sirf ek baar run.
- **Pre-registration:** protocol test seeds chalne se *pehle* git mein commit hua; commit history order prove karti hai (`sim/PROTOCOL.md`, `sim/PROTOCOL_STUDY2.md`).
- **Multiple comparisons:** ek table mein ~36 comparisons hon toh 95% par ek-do sirf chance se significant dikh sakte hain; akele marginal cells ko effect nahi bola.

B17. Study 1 — design, result, wajah

- **Arms (ablation):** S0 fixed 30 s; A0 actuated round-robin; S1 Raza-style (sirf D); S2 +Q; S3 +F; **S4 +S (proposed)**; NULL (S4 with S = 0); S4X (S ki jagah X); S3S, S3X (F ki jagah S ya X).

- **Factors:** do timing rules (bands, actuated) × do density normalisers (physical: kabhi saturate nahi; saturating: 10 vehicles, video jaisa) × exact aur vision-noise sensors.
- **Result:** X ya S jodne se mean delay maximum ±0.5 s badla; kahin bhi improvement nahi. Decision analysis: S ne 40 mein se 1 run mein chuna hua approach badla.
- **Wajah 1:** decision ke time har red road ki queue badh rahi hoti hai, aur sabse lambi queue (sabse bada X) sabse zyada vehicles (sabse bada D) bhi hoti hai: jo term ranking se agree kare, woh argmax nahi badal sakta.
- **Wajah 2:** X bhi saturate hota hai: jab queue visible road bhar de, over-saturation mein har full road ka X = 1.
- **Jo matter kiya:** actuated timing fixed-time ke comparison mein delay aadha karti hai; unequal over-saturated demand mein 300.6 → 33.8 s. Raza-style bands zyada tar scenarios mein fixed-time se bhi kharab, kyunki har green kam se kam 30 s aur kabhi jaldi khatam nahi hota.



Study 1: timing rule vs score composition.

B18. Study 2 — final enhancement

Method:

$$\text{score}_i = D_i + \lambda\left(\frac{1}{a - S_i} - \frac{1}{a}\right), \quad \lambda = 1,\ a = 1.1$$

- *D* har road ke liye ek common normaliser use karta hai: storage-blind, Raza jaisa.
- *S har road ki apni* visible length ke relative hai: storage-aware.
- Term ki value: S = 0 par 0, 0.5 par 0.76, 0.9 par 4.09, 1 par 9.09 — road full hone tak chhoti, phir sabse badi.
- **Timing:** actuated (10–60 s, 2 s passage time) plus *protection*: 20 s green ke baad, agar kisi waiting road ka S ≥ 0.85 hai aur active road se zyada hai, toh green khatam.

- **Source:** Mohajerpoor Eq. 17 ka term $\Lambda/(\alpha\Lambda - \delta)$, Λ se divide = $1/(\alpha - S)$; Eq. 9 constraint $x \le \beta\Lambda$. Mere choices: $-1/\alpha$ shift, λ, α, $\beta = 0.85$, 20 s guard.

Compare kiye methods (yahi ablation hai): FT fixed-time; ACT actuated; CMP capacity-aware max pressure (Gregoire 2015: har 5 s sabse bada vehicles/capacity serve karo); RAZA (bands); RAZA_A (Raza + actuated); PROP_B (+ barrier); **PROP (+ protection)**; PROP_CNT (camera ki jagah stopped count se S); camera noise ke saath PROP.

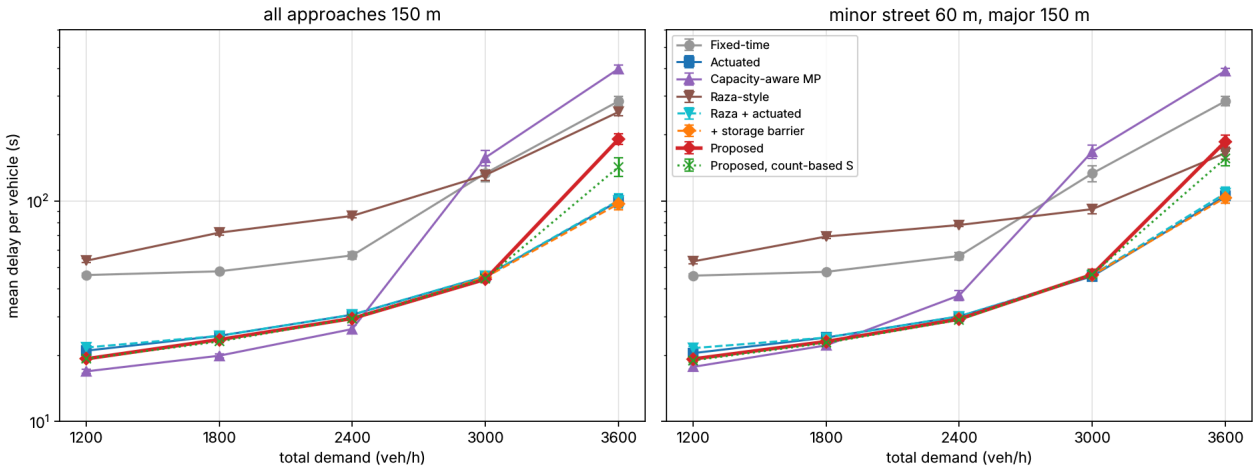
Settings kaise chuni (validation seeds 0–9): pehle runs mein dikha ki protection capacity ke paas spillback aadha karta hai lekin capacity se upar delay kai guna kar deta hai. Teen guards try kiye; selection rule, unke results se pehle likha, kehta tha ki har heavy condition mein delay PROP_B se 5% ke andar ho. Koi qualify nahi hua, toh PROP ko sabse kam bure guard (20 s) ke saath freeze kiya aur capacity se upar fail report kiya.

Results (test seeds 200–219):

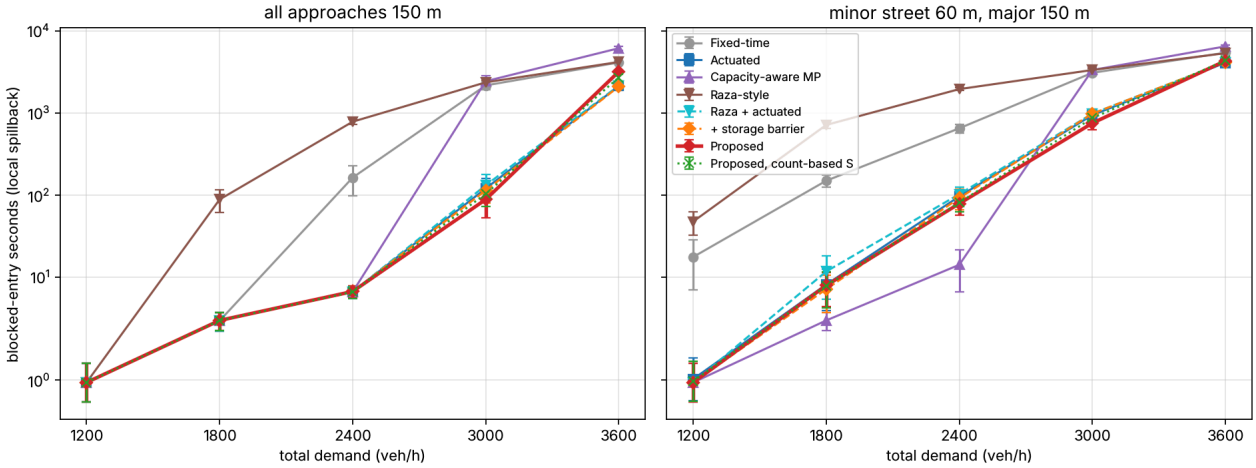
Comparison	Result
vs Raza-style	10 mein se 9 conditions mein delay −34 se −87 s; short-minor 3600 par +21.0 ± 10.5 s
vs actuated, ≤ 2400 veh/h	−0.9 se −1.7 s; **sirf storage term ko credit nahi de sakte** (proposed controller khaali roads bhi skip karta hai aur ek road ko lagatar do baar green nahi deta)
vs actuated, 3000 veh/h	blocked entry −31.9 ± 20.6 s (−26%, uniform), −191.8 ± 73.3 s (−20%, chhoti side street); delay mein significant farak nahi
sirf protection (vs sirf barrier), 3000	−25.3 ± 19.1 s aur −229.2 ± 70.3 s blocked entry
camera vs count, chhoti side street, 3000	−115.4 ± 81.2 s blocked entry
vs actuated, 3600 veh/h	delay +90.9 ± 9.8 s aur +80.7 ± 8.1 s: **fail**
camera errors	2400 tak theek; capacity ke paas chhoti road par benefit zyada tar khatam
capacity-aware max pressure	≤ 1800 veh/h par best delay; ≥ 3000 par collapse (bahut zyada switching)

Capacity se upar fail kyun: har road full ke paas hai, toh koi na koi waiting road hamesha S ≥ 0.85 par hoti hai; greens baar-baar kat-te hain; har switch mein 3 s yellow waste; capacity girti hai; queues badhti hain. Mohajerpoor: queue-formation period mein spillback aksar avoid nahi ho sakta aur constraint ko relax karna padta hai.

Kahan: `sim/study2.py` (`Policy.choose`, `Policy.should_end`), tables `results/sim/study2/tables.md`, protocol `sim/PROTOCOL_STUDY2.md`.



Study 2: demand ke saath mean delay, dono junctions.



Study 2: demand ke saath local spillback (blocked-entry seconds).

B19. Papers — har paper kya kehta hai, kya liya

Paper	Kya karta hai	Liya	Nahi liya, aur kyun
Raza et al., IEEE Access 2025	YOLO + PCE density (Eq. 1) × lane priority; Green Denial Counter; 40/60/120 s bands; SUMO aur real footage	Baseline structure	Lane priority (camera approaches turn ke hisaab se split nahi); exact bands
Li, Lu & Wang, IEEE T-ITS 2025	Connected-vehicle trajectories se queue profile; corridor MILP; under/over-saturation test	Idea: queue space mein profile ki tarah	Trajectories aur solver chahiye; offline fixed-time
Wei et al., IEEE T-ITS 2025	Link-transmission queue dynamics ke saath hierarchical MPC; spillback se pehle action	Idea: queue kidhar ja rahi hai us par action (S)	State estimation maan li; network solver

Paper	Kya karta hai	Liya	Nahi liya, aur kyun
Mohajerpoor, Cai & Ramezani, IEEE T-ITS 2023	FASC: over-saturated isolated junction ke liye optimal cycles/splits; shockwave queue model; spillback constraint $x \leq \beta\Lambda$ (Eq. 9); penalty $\Lambda/(\alpha\Lambda - \delta)$ (Eq. 17); Aimsun mein fixed/actuated/CMP se 63/55/40% kam delay	Constraint aur penalty ka shape; CMP baseline	Unka optimiser aur demand prediction; model se estimated queue
Gregoire et al., IEEE TCNS 2015	Capacity-aware backpressure	Baseline (CMP)	—
Varaiya, Transp. Res. C 2013	Max pressure, unlimited storage maanta hai	Context	—

B20. Kya mera hai aur kya borrowed

Borrowed (cite kiya)	Mera
YOLOv8, ByteTrack	Queue axis, size-scaled 1 s stopped test, contiguous queue tail X, risk S
Raza ki PCE density aur starvation guard	Queue position ko measure karna jo Mohajerpoor model karte hain
Mohajerpoor ke constraint aur penalty ke shapes	Camera controller ke andar online rule; 20 s guard; 2 s passage-time fix
Actuated control, max pressure (baselines)	Audit, dono studies, null aur count controls, noise test, pre-registration

Sir ke liye analogy: T-DLcR = LcR + ek borrowed noise filter, ek naye step mein; contribution combination, adaptation aur evidence hai.

B21. Limitations aur future work

Limitation	Asar	Agla step
X sirf visible road cover karta hai; metres mein nahi	X = 1 matlab “visible hissa full”; roads visible storage ke share se compare hoti hain	Homography se metres (road ke chaar known points)
Door ke vehicles miss (North, East)	Wahan X underestimate	Door ki roads par higher-resolution inference ya crops; labelled frames par YOLO fine-tune
X ki hand-labelled accuracy nahi	Accuracy sirf proxies aur images se dikhayi	`audit/label_queue_tail.py` + `audit/score_queue_tail.py`
Ek hi junction, isolated	Spillback ka cost sirf waiting, upstream blocking nahi	Do-junction SUMO corridor
Capacity se upar protection fail	Saari roads full hon toh delay double	Regime switch: jab har road full ke paas ho, protection band

Limitation	Asar	Agla step
Weights aur thresholds haath se set	Optimal nahi	Sirf validation seeds par tune

Part C — Cross-question bank

Q1. Ek line mein aapka contribution kya hai? Detections aur tracks se queue kahan khatam hoti hai, iska calibration-free measurement, jise signal timing mein storage limit ki tarah use kiya, aur ek pre-registered test jo dikhata hai ki yeh capacity ke paas spillback kam karta hai aur capacity se upar fail hota hai.

Q2. Raza ko hi kyun chuna? Yeh sabse kareeb ka recent camera-only controller hai: YOLO, PCE density, rule-based, deployed. Extend karne mein simple hai aur iski limitation clear hai: count wali state.

Q3. "Reactive, count-only" kya Raza ki apni batayi limitation hai? Nahi. Unki batayi limitations hardware, continual learning, label noise aur sensor fusion hain. Count-only wali limitation unke algorithm ka mera analysis hai, aur main yeh clearly kehta hoon.

Q4. Kya aapne Raza ko reproduce kiya? Nahi. Raza-style baseline: same structure, scaled bands, lane-priority weights nahi.

Q5. Aapne model train kyun nahi kiya? Contribution detector nahi hai. COCO-pretrained YOLO yeh classes pehle se detect karta hai, aur is camera ka labelled data mere paas nahi hai. Detection quality door par matter karti hai, jo maine measure kiya (YOLOv8n vs m) aur future work mein rakha.

Q6. Box ka bottom-centre hi kyun? Wahan vehicle road ko chhoota hai; lambe vehicle ke box ka centre road ke upar hota hai aur galat region mein gir sakta hai.

Q7. 0.2 box heights per second kyun? Ek second kyun? Apne size ka 0.2 per second, car ke liye lagbhag 1 m/s — practically ruki hui. Ek second box jitter ko average karta hai. Dono real frames par audit ke dauraan fix hue; saath mein false stops 4.28 se 0.76 per vehicle kiye.

Q8. Box height se divide kyun? Perspective. Door ki tez chalti car kam pixels cover karti hai; apne size se divide karne se paas aur door ki cars same scale par aati hain.

Q9. Queue contiguous kyun honi chahiye? Do car-length kyun? Parked car ya door ki koi galat detection queue ka hissa nahi hai aur use X set nahi karna chahiye. Do car-length mein ek vehicle ki length ka gap plus spacing aa jaati hai. Road ki direction mein measure karte hain kyunki side se dikhti cars height se zyada chaudi hoti hain.

Q10. Kya X metres mein hai? Nahi. Yeh drawn line ke saath visible road ka fraction hai. Perspective door ke hisse ko compress karta hai, aur road image ke bahar bhi jaati hai. Homography se metres mein convert ho sakta hai.

Q11. North aur East door hain aur cut-off hain — wahan X sahi kaise? X har road ke visible hisse ke relative hai, toh har road ke liye consistent hai, lekin door ki cars chhoti hain aur kuch miss hoti hain, isliye wahan X underestimate hota hai. Yeh measurement ki main limitation hai, aur control tests simulator mein hain jahan lengths exact hain.

Q12. S ke liye 5 second kyun, aur slope ke liye 2.5 s kyun? 5 s mein lagbhag kuch vehicles queue mein jud jaate hain — overflow anticipate karne ke liye kaafi. 2.5 s ke points slope ko robust banate hain; purana 0.5 s window + 5 s horizon noise ko das guna kar deta tha.

Q13. S = 1 ka matlab kya hai? Kya yeh spillback hai? 5 s mein queue visible road bharne wali hai. Yeh local storage-exhaustion risk hai; downstream spillback ke liye woh road dikhni chahiye jismein vehicles ja rahe hain.

Q14. Prove karo ki count forecast blind hai. Strip mein 4 vehicles aate hain. Full hone ke baad har frame Q = 1, slope 0, F = 1 + 0 = 1, “steady”, chahe peeche kitni bhi tezi se vehicles judte rahein. X peeche khisakta hai, toh S badhta hai.

Q15. Kya woh blind spot aapke video mein sach mein hua? Kam: Q = 1 sirf 0.6% frames tak. Yeh mathematically sahi hai aur simulation mein test hua, aur main claim nahi karta ki footage yeh demonstrate karti hai.

Q16. Score ke weights kaise chune? Haath se, aur jaan-boojh ke tune nahi kiye, taaki test data par fit na ho. Weight 0.3 par S ne chaalis mein ek decision badla; itna bada weight jo farak laaye, score ko zyada tar S bana dega, jo khud bhi saturate hota hai.

Q17. Video par control evaluate kyun nahi kar sakte? Recorded vehicles filmed light follow karte hain. “Vehicles served” un recorded departures ko ginta tha jo mere simulated green mein pade: “best” controller ne poore clip North green rakha kyunki wahin asli light ne traffic chhoda tha.

Q18. +21% kya tha, aur aapne kaise prove kiya ki fake tha? S3 → S4 mein do cheezein badli: S add hua aur base weight 0.7 se 0.4 hua. Null control — S4 jismein S ko 0 kar diya — ne bhi exactly 158 vehicles diye. Gain re-weighting se aaya, S se nahi.

Q19. SUMO kyun? Kya simulation unrealistic nahi? Yeh closed loop hai: vehicles mere signal par react karte hain, jo causal comparison ke liye zaroori hai. Is field mein standard hai; Raza ne bhi SUMO use kiya. Realism ki limits (ek junction, simple demand) list ki hain.

Q20. Kaise pata simulation aapka hi controller test karta hai, re-implementation nahi? `sim/closed_loop.py` controller, sequencer, predictor aur score seedha `src/` se import karta hai; sirf sensor alag hai, aur woh bhi same functions use karta hai.

Q21. Confidence interval kya hai aur paired kyun? Ek range jismein 95% confidence se true mean difference hai. Har method same 20 random demand realisations par chalta hai, toh main per-seed differences leta hoon, jo seed-to-seed ka bada variation hata deta hai.

Q22. Kya aapne test data par tune kiya? Nahi. Settings validation seeds par pehle se likhe rule se chuni; protocol test seeds se pehle commit hua, aur git history order dikhati hai.

Q23. Study 1 fail hua. Phir bhi woh count kyun ho? Assignment yeh test karna hai ki enhancement help *karta hai ya nahi*. Negative result pre-registered aur explained hai, aur usi explanation ne study 2 design kiya.

Q24. Kya study 2 bas Mohajerpoor hi hai? Nahi. Woh queue position model karte hain aur predicted demand ke saath poore cycles optimise karte hain. Main use video se measure karta hoon aur camera controller ke andar per-step rule mein use karta hoon. Shapes unke hain aur cite kiye; measurement, controller design aur evidence mere hain. Count-based control dikhata hai ki chhoti roads par measurement khud matter karta hai.

Q25. $1/(\alpha - S)$ kyun? $\alpha = 1.1$ kyun? Mohajerpoor ki penalty queue ke scaled link length ke paas aate hi bahut badh jaati hai; chhoti queues ko ignore karti hai aur full ke paas dominate karti hai. $\alpha = 1.1$ se $S = 1$ par bhi value finite rehti hai (9.09).

Q26. $\beta = 0.85$ aur 20 s guard kyun? Validation seeds par chune. Protection ko road full hone se pehle act karna chahiye (0.85), aur guard greens ko bahut chhota katne se rokta hai.

Capacity se upar koi guard mera pehle se set criterion pass nahi kar paaya; 20 s sabse kam bura tha, aur main failure report karta hoon.

Q27. Raza par zyada gain actuated timing se hai. Aapke liye kya bacha? Actuated control (stronger baseline) ke against, mera contribution capacity ke paas same delay par 20–26% kam spillback hai, aur camera vs count result.

Q28. Capacity ke neeche aap actuated ko 1–2 s se beat karte ho — kya yeh aapka term hai? Prove nahi hota. Mera controller khaali roads skip karta hai aur ek road ko repeat nahi karta, aur koi ablation arm in habits ko barrier term se alag nahi karta. Main sirf capacity ke paas spillback reduction claim karta hoon.

Q29. Capacity se upar fail kyun hota hai? Har road full ke paas hai, toh rule baar-baar greens kaat-ta hai; har switch 3 s yellow waste karta hai, capacity girti hai, queues badhti hain. Fix: regime switch — jab saari roads full hon toh protection relax karo, jaisa Mohajerpoor kehte hain.

Q30. Light traffic par capacity-aware max pressure aapse better hai — use kyun nahi use karte? Woh har 5 s re-decide karta hai, chhoti queues ke liye ideal, lekin heavy demand mein lagatar switch karta hai aur yellows mein capacity kho deta hai (3000 veh/h par 157 s vs 44 s delay). Main use light demand par best report karta hoon.

Q31. Kya camera sach mein zaroori hai, ya count se kaam chal jaata? Same rule, stopped count se S: zyada tar conditions mein barabar; chhoti road par capacity ke paas camera ka measured tail 115 s kam spillback deta hai. Jahan queues road ke comparison mein lambi hon, wahan spatial measurement matter karta hai.

Q32. Aapka ablation kya hai? Measurement: legacy → robust stopped test → redrawn geometry. Study 1: S0 → S1 → S2 → S3 → S4, plus NULL aur swap controls, do timing rules ke saath. Study 2: Raza → + actuated → + barrier → + protection → camera vs count → camera noise.

Q33. X kitna accurate hai? Proxies (false stops, jump rate) aur images se dikhaya; abhi hand-labelled error nahi. `audit/label_queue_tail.py` aur `audit/score_queue_tail.py` frames label hone ke baad mean absolute error aur road ke 10% ke andar wale share dete hain.

Q34. Teen mahine aur milein toh kya karoge? Hand-labelled accuracy; homography se metres; door ki roads par better detection; regime switch; spillback ka upstream cost measure karne ke liye do-junction corridor.

Part D — Reference

D1. Yaad karne wale numbers

Number	Matlab
4.28 → 0.76	False stops per vehicle, purana → robust stopped test
7.3% → 0.06%	North frames jismein X road ke 0.3 se zyada jump kare
4 of ~9, 3 of ~10	Door North / East par YOLOv8m ki dhoondhi queued cars
158 = 158	S4 aur NULL control: +21% ka artefact
±0.5 s, 1 in 40	Study 1: score mein X/S ka asar; decisions badle
300.6 → 33.8 s	Study 1: fixed-time vs actuated, unequal over-saturated
53.6 → 19.2 s	Study 2: Raza vs proposed, uniform 1200 veh/h
121 → 89 s (−26%)	Study 2: spillback, actuated vs proposed, uniform 3000
945 → 753 s (−20%)	Study 2: spillback, chhoti side street 3000
−115 s	Camera vs count, chhoti side street 3000
99.9 → 190.8 s	Study 2: capacity se upar actuated vs proposed — failure
20 seeds, 95% CI	Har comparison

D2. Har formula kahan se aaya

Formula	Source	Yahan chuna gaya
$D = \Sigma$ count × PCE	Raza Eq. 1	saturation_count se normalisation
Q	Queue-based control (common)	Strip aur capacity
$\alpha D + (1-\alpha)Q$	Original codebase	$\alpha = 0.5$
F, S = value + slope × horizon	Wei ka idea; least squares (standard)	3 s, 5 s, 2.5 s window
Stopped test	Yeh project	0.2, 1 s
X (contiguous chain)	Yeh project; Li ka profile idea	2 car lengths
Score weights	Yeh project	Haath se, tune nahi
Barrier $1/(\alpha - S) - 1/\alpha$	Mohajerpoor Eq. 17	$-1/\alpha$ shift, $\lambda = 1$, $\alpha = 1.1$
Protection $S \geq \beta$	Mohajerpoor Eq. 9	$\beta = 0.85$, 20 s guard
Actuated gap-out	Signal Timing Manual	2 s passage time
n/C	Gregoire 2015	Baseline

Formula	Source	Yahan chuna gaya
Paired t-interval	Standard statistics	—

D3. Code map

Stage	File · function
Detection, tracking	`src/tracking.py`; cache `src/track_cache.py`
Regions, axis, position	`src/lane_analysis.py` · `ApproachAssigner`, `_polyline_fraction`
D, Q, stopped, X	`src/traffic_metrics.py` · `MetricsEngine.update`, `_is_stopped_windowed`, `queue_tail_reach_scaled`
F, S	`src/traffic_metrics.py` · `QueuePredictor`
Score	`src/traffic_metrics.py` · `compute_score_weighted`
Controller, timing	`src/signal_controller.py` · `AdaptiveController`, `PhaseSequencer`
Study 1	`sim/closed_loop.py`, `sim/experiment.py`, `sim/PROTOCOL.md`
Study 2	`sim/study2.py`, `sim/study2_figures.py`, `sim/PROTOCOL_STUDY2.md`
Audit	`audit/*.py`, `AUDIT_REPORT.md`

D4. Live dikhane ke commands

```
python audit/explain_frame.py --frame 2036 --approach West     # ek frame ke saare numbers
python audit/explain_frame.py --find --approach South          # haath se solve karne ke
liye naya frame
python -m src.main control --video videos/bellevue_116th_busy.mp4 --config
config/final/S4.json \
  --track-cache results/track_cache/bellevue_116th_busy__yolov8m__c0p30.json.gz --overlay
demo
python audit/label_queue_tail.py --clip busy                   # asli queue end label karo
python audit/score_queue_tail.py --clip busy                   # X ki accuracy
python -m pytest -q                                            # 800+ tests
```

D5. Short glossary

Term	Matlab
Ablation	Ek-ek component add/remove karke dekhna ki har ek kya contribute karta hai
Actuated control	Green tab khatam jab detector zone passage time tak khaali rahe
Argmax	Sabse badi value wala option chunna
Closed / open loop	Output input ko affect karta hai / nahi karta (simulated vs recorded vehicles)

Term	Matlab
COCO	80 object classes wala bada public image dataset, YOLO pretrain karne ke liye
FASC	Mohajerpoor ka optimal signal control algorithm
Homography	Image pixels se ground plane par metres mein mapping
Least squares	Woh line fit karna jo points se squared error minimum kare
Max pressure	Jis road ka queue “pressure” sabse zyada, use serve karo
MPC	Model Predictive Control: model se aage plan karo, pehla step apply karo, repeat
Null control	Same change, lekin test hone wala ingredient band karke
Pre-registration	Test chalane se pehle experiment plan likh dena
Shockwave model	Traffic-flow theory jo batati hai queue ka peechhla hissa kaise move karta hai
SUMO	Simulation of Urban MObility
T-ITS	IEEE Transactions on Intelligent Transportation Systems